

enabling models to conditionally generate reasoning trajectories that approximate those observed during training. As such, the effectiveness of CoT reasoning is fundamentally governed by the nature and degree of distribution discrepancy between training data and test queries (rather than by model architecture or scale).

To formalize this view, we first introduce notation for the training and test distributions. Let $\mathcal{D}_{\text{train}}$ denote the training distribution over input-output pairs (x, y) , where x represents a reasoning problem and y denotes the solution sequence (including intermediate reasoning traces). During training, the model learns a parametric mapping $f_\theta(x) \approx y$ by minimizing the empirical training risk

$$\hat{R}_{\text{train}}(f_\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i) \quad (1)$$

where $(x_i, y_i) \sim \mathcal{D}_{\text{train}}$ are i.i.d. samples and ℓ is a loss function (e.g., cross-entropy). The corresponding expected (population) training risk is

$$R_{\text{train}}(f_\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{train}}} [\ell(f_\theta(x), y)] \quad (2)$$

At inference time, given a test query sampled from a potentially different distribution $\mathcal{D}_{\text{test}}$, the model generates a response. The expected test risk is

$$R_{\text{test}}(f_\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{test}}} [\ell(f_\theta(x), y)] \quad (3)$$

Definition 3.1 (Distribution Discrepancy). *Given training distribution $\mathcal{D}_{\text{train}}$ and test distribution $\mathcal{D}_{\text{test}}$, we define the distribution discrepancy as*

$$\Delta(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}) := \text{TV}(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}) \quad (4)$$

where $\text{TV}(P, Q)$ is the total variation distance,

$$\text{TV}(P, Q) := \sup_A |P(A) - Q(A)| = \frac{1}{2} \int |dP - dQ| \quad (5)$$

Theorem 3.1 (Generalization Bound). *Assume the loss is bounded, i.e., for all (x, y) , $0 \leq \ell(f_\theta(x), y) \leq B$. Let $\{(x_i, y_i)\}_{i=1}^n$ be i.i.d. samples from $\mathcal{D}_{\text{train}}$ and let $\hat{R}_{\text{train}}(f_\theta)$ be the empirical training risk defined above. Then for any $\delta \in (0, 1)$, with probability at least $1 - \delta$ over the draw of the training sample, the expected test risk satisfies*

$$R_{\text{test}}(f_\theta) \leq \hat{R}_{\text{train}}(f_\theta) + 2B \Delta(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}) + B \sqrt{\frac{\log(1/\delta)}{2n}}. \quad (6)$$

The proof is provided in Appendix H.1.

Theorem 3.1 provided a theoretical foundation for the data distribution lens. Guided by it, we identify three critical dimensions along which distribution shifts can occur: task, length, and format.

$$\Delta(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}) = \Phi(\Delta_{\text{task}}, \Delta_{\text{length}}, \Delta_{\text{format}}) \quad (7)$$

where Φ is a monotonically increasing composition function that aggregates all discrepancies. Δ_{task} , Δ_{length} , and Δ_{format} measure the distribution discrepancy introduced by unseen tasks, various lengths, and prompt format variants.

4. DataAlchemy: A Controllable Environment

To empirically validate the data distribution lens, we introduce DATAALCHEMY, an abstract, controllable, and clean environment. It distills real-world NLP tasks into basic *atoms*, *elements*, and *transformations* as illustrated in Figure 2.

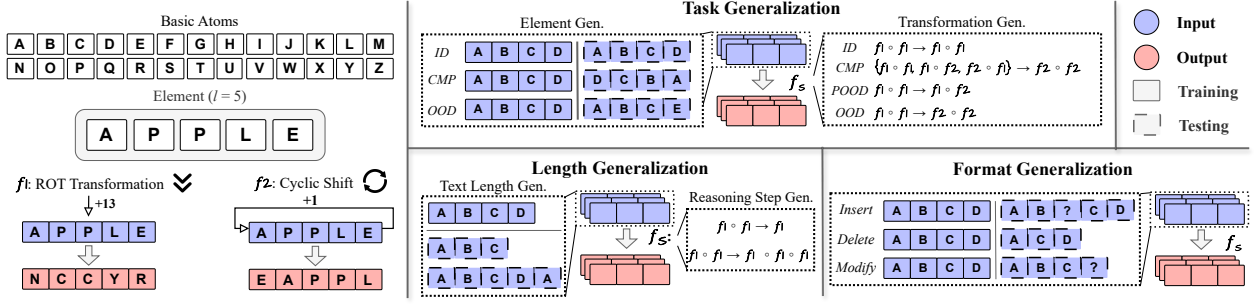


Figure 2 | Framework of DATAALCHEMY. DATAALCHEMY provides an abstract representation system that distills various real-world NLP tasks into key components: *atoms*, *elements*, and *transformations*. By varying these components, we curate data that exhibits various distribution discrepancies following *task*, *length*, and *format* generalization. DATAALCHEMY achieves full and fine-grained control over the entire evaluation pipeline. Later, we train models *from scratch* to avoid data leakage and employ controlled experiments to rigorously test the hypotheses.

4.1. Basic Atoms and Elements

We abstract tokens in the real-world NLP tasks into basic *atoms* represented by an alphabet of 26 $\mathcal{A} = \{A, B, C, \dots, Z\}$. Based on *atoms*, we further construct *element* $\mathbf{e}$ as an ordered sequence of atoms with length l , reflecting the text space (considering the text consists of tokens):

$$\mathbf{e} = (a_0, a_1, \dots, a_{l-1}) \quad \text{where } a_i \in \mathcal{A}, l \in \mathbb{Z}^+ \quad (8)$$

Note that we can construct at most $|\mathcal{A}|^l$ distinct elements, which provides a versatile approach for data curation by manipulating element length l .

4.2. Transformations

Similarly, we abstract operations LLM performed on text in the real world (e.g., summarize, paraphrase, and reasoning) as *transformations* that operate on elements $F : \mathbf{e} \rightarrow \hat{\mathbf{e}}$. In this work, we mainly instantiate two fundamental *transformations*: the ROT Transformation and the Cyclic Position Shift. Additional *transformations* are considered in the Appendix F.1 to avoid bias. To formally define the *transformations*, we introduce a bijective mapping $\phi : \mathcal{A} \rightarrow \mathbb{Z}_{26}$, where $\mathbb{Z}_{26} = \{0, 1, \dots, 25\}$, such that $\phi(c)$ maps a character to its zero-based alphabetical index.

Definition 4.1 (ROT Transformation). Given an element $\mathbf{e} = (a_0, \dots, a_{l-1})$ and a rotation parameter $n \in \mathbb{Z}$, the ROT Transformation f_{rot} produces an element $\hat{\mathbf{e}} = (\hat{a}_0, \dots, \hat{a}_{l-1})$. Each atom $\hat{a}_i$ is:

$$\hat{a}_i = \phi^{-1}((\phi(a_i) + n) \pmod{26}) \quad (9)$$

This operation cyclically shifts each atom n positions forward in alphabetical order. For example, if $\mathbf{e} = (A, P, P, L, E)$ and $n = 13$, then $f_{\text{rot}}(\mathbf{e}, 13) = (N, C, C, Y, R)$.

Definition 4.2 (Cyclic Position Shift). Given an element $\mathbf{e} = (a_0, \dots, a_{l-1})$ and a shift parameter $n \in \mathbb{Z}$, the Cyclic Position Shift f_{pos} produces an element $\hat{\mathbf{e}} = (\hat{a}_0, \dots, \hat{a}_{l-1})$. Each atom $\hat{a}_i$ is defined by a cyclic shift of indices:

$$\hat{a}_i = a_{(i-n) \pmod{l}} \quad (10)$$

This transformation cyclically shifts the positions of the atoms within the sequence by n positions to the right. For instance, if $\mathbf{e} = (A, P, P, L, E)$ and $n = 1$, then $f_{\text{pos}}(\mathbf{e}, 1) = (E, A, P, P, L)$.

Definition 4.3 (Generalized Compositional Transformation). *To model multi-step reasoning, we define a compositional transformation as the successive application of a sequence of operations. Let $S = (f_1, f_2, \dots, f_k)$ be a sequence of operations, where each f_i is one of the fundamental transformations $\mathcal{F} = \{f_{\text{rot}}, f_{\text{pos}}\}$ with its respective parameters. The compositional transformation f_S for the sequence S is the function composition:*

$$f_S = f_1 \circ f_2 \circ \dots \circ f_k \quad (11)$$

The resulting element $\hat{\mathbf{e}}$ is obtained by applying the operations sequentially to an initial element $\mathbf{e}$:

$$\hat{\mathbf{e}} = f_k(f_{k-1}(\dots(f_1(\mathbf{e}))\dots)) \quad (12)$$

This design enables the construction of arbitrary transformations with the type, parameters, order, and length. At the same time, we can naturally acquire the CoT reasoning step by decomposing the intermediate process:

$$\underbrace{f_S(\mathbf{e})}_{\text{Query}} : \underbrace{\mathbf{e} \xrightarrow{f_1} \mathbf{e}^{(1)} \xrightarrow{f_2} \mathbf{e}^{(2)} \dots \xrightarrow{f_{k-1}} \mathbf{e}^{(k-1)} \xrightarrow{f_k}}_{\text{Reasoning traces}} \underbrace{\boxed{\hat{\mathbf{e}}}}_{\text{Answer}} \quad (13)$$

Illustrative examples of atoms, elements, and transformations are detailed in Appendix B.

4.3. Environment Setting

Through systematic manipulation of elements and transformations, DATAALCHEMY, we can train and probe various LLMs under various tasks, lengths, and format distributions. In the controlled experiment, we employ decoder-only LLMs with GPT and LLaMA architectures and parameter sizes ranging from 62K to 3B when training from scratch. In the real-world experiments, we utilize two state-of-the-art (SOTA) LLMs: LLaMA3-8B-Instruct [Dubey et al. \(2024\)](#) and Qwen3-14B [Yang et al. \(2025\)](#). We construct elements with 2 to 6 basic atoms, which produce 676 to 308,915,776 data samples. We initialize the two transformations $f_1 = f_{\text{rot}}(e, 13)$ and $f_2 = f_{\text{pos}}(e, 1)$. We consider both hard metrics, i.e., exact match rate, and soft metrics, i.e., Levenshtein distance (edit distance) ([Yujian and Bo, 2007](#)), and BLEU score ([Papineni et al., 2002](#)) for evaluation. To enable a fine-grained analysis, we evaluate reasoning traces, the final answer, and the full chain in the LLM response. Detailed environment setting and implementation are provided in Appendix D.

5. Task Generalization

To investigate the extent to which CoT reasoning can handle *tasks* under various distribution discrepancies, we design task generalization experiments. As we discussed in Section 4, we decompose tasks into a combination of various *transformations* and *elements*. Therefore, we consider task generalization from two dimensions: transformation generalization and element generalization.

5.1. Transformation Generalization

Experiment setup. To formulate different distribution discrepancies for task generalization, we design the following progressive scenarios based on the proposed measurement (detailed in Appendix E.1). (i) In-Distribution (ID). The transformations in the test set are identical to those observed during training, e.g., $f_1 \circ f_1 \rightarrow f_1 \circ f_1$. (ii) Composition (CMP). Test samples comprise novel compositions, where basic transformations are observed during training, e.g., $f_1 \circ f_1, f_1 \circ f_2, f_2 \circ f_1 \rightarrow f_2 \circ f_2$. (iii)